6710503739_HW#8_Exercise-3

September 6, 2026

[1]:
```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

pd.set_option("display.width", 88)
pd.set_option("display.max_rows", 120)

df = pd.read_csv("bank-data.csv")
print("Shape:", df.shape)
print("Missing values:", df.isna().sum().sum())
df.head()
```

```
Shape: (600, 12)
Missing values: 0
```

[1]:

	id	age	sex	region	income	married	children	car	save_act	current_act	mortgage	pep
0	ID12101	48	FEMALE	INNER_CITY	17546.0	NO	1	NO	NO	NO	NO	YES
1	ID12102	40	MALE	TOWN	30085.1	YES	3	YES	NO	YES	YES	NO
2	ID12103	51	FEMALE	INNER_CITY	16575.4	YES	0	YES	YES	YES	NO	NO
3	ID12104	23	FEMALE	TOWN	20375.4	YES	3	NO	NO	YES	NO	NO
4	ID12105	57	FEMALE	RURAL	50576.3	YES	0	NO	YES	NO	NO	NO

1 Part A — Data Preparation

1.1 A1. Attribute Selection

Attribute	Reason
`age`	Life stage drives investment behaviour.
`income`	Direct measure of the capacity to invest.
`married`	Family connect to financial planning.
`children`	change how much money to invest.
`car`	disposable assets.

Attribute	Reason
`save_act`	Already saving = already an investing customer.
`mortgage`	dept reduces free cash for investing.
`pep`	Target variable.

removed

Removed	Reason
`id`	carries no behavioural meaning.
`sex`	50% support that join almost every itemset without explaining anything.
`region`	4 unbalanced levels and drags rare items below the support threshold.
`current_act`	75.8% YES. It would appear in nearly every frequent itemset.

[3]:
```
SELECTED = ["age", "income", "married", "children", "car", "save_act",␣
 ↪"mortgage", "pep"]
REMOVED  = ["id", "sex", "region", "current_act"]

data = df[SELECTED].copy()
data.head()
```

[3]:

	age	income	married	children	car	save_act	mortgage	pep
0	48	17546.0	NO	1	NO	NO	NO	YES
1	40	30085.1	YES	3	YES	NO	YES	NO
2	51	16575.4	YES	0	YES	YES	NO	NO
3	23	20375.4	YES	3	NO	NO	NO	NO
4	57	50576.3	YES	0	NO	YES	NO	NO

1.2 A3. Discretization of Numerical Attributes

Attribute	Method	Groups
`age`	Equal-frequency (3 quantiles)	YOUNG / MIDDLE / SENIOR
`income`	Equal-frequency (3 quantiles)	LOW / MEDIUM / HIGH
`children`	Domain-based binning	0 / 1-2 / 3+

Why this method

- Equal-frequency (quantile) binning for `age` and `income`. Income is right-skewed Equal-width bins would put most customers in the "low" bin and leave "high" almost empty,Equal-frequency binning gives ~200 customers per group, so all three items can pass the support threshold and be compared fairly.
- Domain binning for `children`. It is a small discrete count, so the groups follow real household meaning: no dependants / small family / large family.

Why the grouping makes sense - the groups match how a bank actually segments customers: young/middle/senior maps to the accumulate → invest → preserve life cycle; low/medium/high income maps to investment capacity; and family size maps to how much surplus income is left over.

[4]:
```
age_bins    = pd.qcut(df.age, 3, retbins=True)[1].round(0)
income_bins = pd.qcut(df.income, 3, retbins=True)[1].round(1)

data["age"]      = pd.qcut(df.age, 3, labels=["YOUNG", "MIDDLE", "SENIOR"])
data["income"]   = pd.qcut(df.income, 3, labels=["LOW", "MEDIUM", "HIGH"])
data["children"] = pd.cut(df.children, [-1, 0, 2, 10], labels=["0", "1-2",␣
 ↪"3+"])

pd.concat([df[["age", "income", "children"]].add_suffix("_before").head(),
           data[["age", "income", "children"]].add_suffix("_after").head()],␣
 ↪axis=1)
```

[4]:

	age_before	income_before	children_before	age_after	income_after	children_after
0	48	17546.0	1	MIDDLE	LOW	1-2
1	40	30085.1	3	MIDDLE	MEDIUM	3+
2	51	16575.4	0	SENIOR	LOW	0
3	23	20375.4	3	YOUNG	MEDIUM	3+
4	57	50576.3	0	SENIOR	HIGH	0

1.3 A2. Convert Categorical Attributes to Binary (One-Hot Encoding)

Transformed
married_YES = 1, married_NO = 0
married_NO = 1, married_YES = 0
income_HIGH = 1, income_LOW = 0, income_MEDIUM = 0
pep_YES = 1, pep_NO = 0

[5]:
```
onehot = pd.get_dummies(data, prefix_sep="_")
onehot.head(3).astype(int).T
```

[5]:

	0	1	2
age_YOUNG	0	0	0
age_MIDDLE	1	1	0
age_SENIOR	0	0	1
income_LOW	1	0	1
income_MEDIUM	0	1	0
income_HIGH	0	0	0

```
married_NO      1  0  0
married_YES     0  1  1
children_0      0  0  1
children_1-2    1  0  0
children_3+     0  1  0
car_NO          1  0  0
car_YES         0  1  1
save_act_NO     1  1  0
save_act_YES    0  0  1
mortgage_NO     1  0  1
mortgage_YES    0  1  0
pep_NO          0  1  1
pep_YES         1  0  0
```

1.4 A4. Final Transaction Format

Each customer becomes one transaction = the set of items whose binary value is 1.

[6]:
```
transactions = [set(onehot.columns[row]) for row in onehot.to_numpy().
 ↪astype(bool)]

for i in range(5):
    ordered = [c for c in onehot.columns if c in transactions[i]]
    print(f"T{i+1} ({df.id[i]}): {{" + ", ".join(ordered) + "}")
```

```
T1 (ID12101): {age_MIDDLE, income_LOW, married_NO, children_1-2, car_NO,
save_act_NO, mortgage_NO, pep_YES}
T2 (ID12102): {age_MIDDLE, income_MEDIUM, married_YES, children_3+, car_YES,
save_act_NO, mortgage_YES, pep_NO}
T3 (ID12103): {age_SENIOR, income_LOW, married_YES, children_0, car_YES,
save_act_YES, mortgage_NO, pep_NO}
T4 (ID12104): {age_YOUNG, income_MEDIUM, married_YES, children_3+, car_NO,
save_act_NO, mortgage_NO, pep_NO}
T5 (ID12105): {age_SENIOR, income_HIGH, married_YES, children_0, car_NO,
save_act_YES, mortgage_NO, pep_NO}
```

2 Part B - Association Rule Mining

2.1 B1. Parameter Selection

Parameter	Chosen value
Minimum support	0.10
Minimum confidence	0.70

Why these values are appropriate

- Support 0.10 - 60 customers is a usable target group, while still being low enough to keep 3- and 4-item profiles above the threshold.

- Confidence 0.70 — the base rate of `pep_YES` is 45.7%, so a rule at 70%+ confidence is better than guessing.

What happens if the values are too high or too low

	Too high	Too low
Support	Only trivial 1- and 2-item patterns survive; the specific customer profiles disappear.	Rules built on a handful of customers, which do not generalise.
Confidence	Very few rules; possibly none pointing at `pep_YES`.	Weak rules whose accuracy is no better than the 45.7% base rate.

[7]:
```
MIN_SUP  = 0.10
MIN_CONF = 0.70
```

2.2 B2. Frequent Itemsets

[15]:
```
frequent = apriori(onehot, min_support=MIN_SUP, use_colnames=True)
frequent["itemset"] = frequent.itemsets.apply(lambda s: "{" + ", ".
 ↪join(sorted(s)) + "}")
frequent["length"] = frequent.itemsets.apply(len)
print("\nTop 15 frequent itemsets overall")
frequent.nlargest(15, "support")[["itemset", "support"]].round(4).
 ↪reset_index(drop=True)
```

```
Top 15 frequent itemsets overall
```

[15]:

	itemset	support
0	{save_act_YES}	0.6900
1	{married_YES}	0.6600
2	{mortgage_NO}	0.6517
3	{pep_NO}	0.5433
4	{car_NO}	0.5067
5	{car_YES}	0.4933
6	{married_YES, save_act_YES}	0.4617
7	{pep_YES}	0.4567
8	{mortgage_NO, save_act_YES}	0.4500
9	{children_1-2}	0.4483
10	{children_0}	0.4383
11	{married_YES, mortgage_NO}	0.4350
12	{married_YES, pep_NO}	0.4033
13	{pep_NO, save_act_YES}	0.3917
14	{age_YOUNG}	0.3483

[18]:
```
print("--- Top frequent itemsets by size ---\n")
for k in sorted(frequent.length.unique()):
```

```
    top = frequent[frequent.length == k].nlargest(8, "support")
    print(f"Size {k}:")
    for _, r in top.iterrows():
        print(f" {r.itemset:<48} support = {r.support:.4f}")
        print()
```

```
--- Top frequent itemsets by size ---

Size 1:
 {save_act_YES}                                  support = 0.6900

 {married_YES}                                   support = 0.6600

 {mortgage_NO}                                   support = 0.6517

 {pep_NO}                                        support = 0.5433

 {car_NO}                                        support = 0.5067

 {car_YES}                                       support = 0.4933

 {pep_YES}                                       support = 0.4567

 {children_1-2}                                  support = 0.4483

Size 2:
 {married_YES, save_act_YES}                     support = 0.4617

 {mortgage_NO, save_act_YES}                     support = 0.4500

 {married_YES, mortgage_NO}                      support = 0.4350

 {married_YES, pep_NO}                           support = 0.4033

 {pep_NO, save_act_YES}                          support = 0.3917

 {car_YES, save_act_YES}                         support = 0.3483

 {mortgage_NO, pep_NO}                           support = 0.3483

 {car_NO, save_act_YES}                          support = 0.3417

Size 3:
 {married_YES, mortgage_NO, save_act_YES}        support = 0.3067

 {married_YES, pep_NO, save_act_YES}             support = 0.2917

 {married_YES, mortgage_NO, pep_NO}              support = 0.2850
```

```
 {car_YES, married_YES, save_act_YES}              support = 0.2400

 {mortgage_NO, pep_NO, save_act_YES}               support = 0.2367

 {children_0, married_YES, pep_NO}                 support = 0.2350

 {car_YES, mortgage_NO, save_act_YES}              support = 0.2350

 {car_NO, married_YES, save_act_YES}               support = 0.2217

Size 4:
 {married_YES, mortgage_NO, pep_NO, save_act_YES} support = 0.2000

 {children_0, married_YES, pep_NO, save_act_YES}  support = 0.1783

 {children_0, married_YES, mortgage_NO, pep_NO}   support = 0.1733

 {car_YES, married_YES, mortgage_NO, save_act_YES} support = 0.1667

 {car_NO, married_YES, mortgage_NO, pep_NO}       support = 0.1483

 {car_YES, married_YES, pep_NO, save_act_YES}     support = 0.1467

 {car_NO, married_YES, pep_NO, save_act_YES}      support = 0.1450

 {car_NO, married_YES, mortgage_NO, save_act_YES} support = 0.1400

Size 5:
 {children_0, married_YES, mortgage_NO, pep_NO, save_act_YES} support = 0.1217

 {car_YES, married_YES, mortgage_NO, pep_NO, save_act_YES} support = 0.1050

 {car_NO, children_0, married_YES, mortgage_NO, pep_NO} support = 0.1000
```

2.2.1 B2 Answer — Top frequent itemsets

Itemset	Support
{married_YES, save_act_YES}	0.4617
{mortgage_NO, save_act_YES}	0.4500
{married_YES, mortgage_NO}	0.4350
{married_YES, pep_NO}	0.4033
{pep_NO, save_act_YES}	0.3917
{married_YES, mortgage_NO, save_act_YES}	0.3067
{children_0, married_YES}	0.3000
{income_HIGH, save_act_YES}	0.2850

Itemset	Support
{children_1-2, pep_YES}	0.2750
{income_HIGH, pep_YES}	0.1967
{children_1-2, income_HIGH, pep_YES}	0.1450

2.3 B3. Association Rules

[ ]:
```
rules = association_rules(frequent, metric="confidence", min_threshold=MIN_CONF)
fmt = lambda s: ", ".join(sorted(s))
rules["antecedent"] = rules.antecedents.apply(fmt)
rules["consequent"] = rules.consequents.apply(fmt)
rules["rule"] = rules.antecedent + " -> " + rules.consequent
print(f"Total rules with confidence >= {MIN_CONF}: {len(rules)}")

pep_rules = rules[rules.consequent.isin(["pep_YES", "pep_NO"])].copy()
pep_rules = pep_rules.sort_values("lift", ascending=False)
print(f"Rules predicting pep_YES / pep_NO: {len(pep_rules)}")

top = pep_rules[["rule", "support", "confidence", "lift"]].head(15).round(4)
print(top.rename(columns={"support": "sup", "confidence": "conf"}).
  ↪to_string(index=False))
```

```
Total rules with confidence >= 0.7: 215
Rules predicting pep_YES / pep_NO: 26
                                                          rule    sup    conf
lift
            age_SENIOR, children_1-2, income_HIGH -> pep_YES 0.1050 0.9403
2.0590
           children_1-2, income_HIGH, mortgage_NO -> pep_YES 0.1033 0.9394
2.0571
          children_1-2, income_HIGH, save_act_YES -> pep_YES 0.1250 0.9259
2.0276
                        children_1-2, income_HIGH -> pep_YES 0.1450 0.9255
2.0267
                         age_SENIOR, children_1-2 -> pep_YES 0.1250 0.7979
1.7472
           age_SENIOR, children_1-2, save_act_YES -> pep_YES 0.1017 0.7821
1.7125
children_0, married_YES, mortgage_NO, save_act_YES -> pep_NO 0.1217 0.9125
1.6794
             children_0, married_YES, save_act_YES -> pep_NO 0.1783 0.8992
1.6549
              children_0, married_YES, mortgage_NO -> pep_NO 0.1733 0.8966
1.6501
      car_NO, children_0, married_YES, mortgage_NO -> pep_NO 0.1000 0.8955
1.6482
           married_NO, mortgage_NO, save_act_YES -> pep_YES 0.1067 0.7442
1.6296
```

```
                          married_NO, mortgage_NO -> pep_YES 0.1533 0.7077
1.5497
                   car_NO, children_0, married_YES -> pep_NO 0.1333 0.8000
1.4724
                           children_0, married_YES -> pep_NO 0.2350 0.7833
1.4417
                age_YOUNG, income_LOW, save_act_YES -> pep_NO 0.1167 0.7778
1.4315
```

[11]:
```
print("=== RULES THAT PREDICT pep_YES (customers likely to BUY) ===\n")
yes = pep_rules[pep_rules.consequent == "pep_YES"]
for i, (_, r) in enumerate(yes.head(8).iterrows(), 1):
    flag = "  <-- confidence >= 0.70 AND lift > 1" if r.confidence >= MIN_CONF␣
 ↪and r.lift > 1 else ""
    print(f"{i}. {r.antecedent}  ->  pep_YES")
    print(f"   support = {r.support:.4f}   confidence = {r.confidence:.4f}  ␣
 ↪lift = {r.lift:.4f}{flag}\n")

print("\n=== RULES THAT PREDICT pep_NO (customers unlikely to buy) ===\n")
no = pep_rules[pep_rules.consequent == "pep_NO"]
for i, (_, r) in enumerate(no.head(5).iterrows(), 1):
    print(f"{i}. {r.antecedent}  ->  pep_NO")
    print(f"   support = {r.support:.4f}   confidence = {r.confidence:.4f}  ␣
 ↪lift = {r.lift:.4f}\n")
```

```
=== RULES THAT PREDICT pep_YES (customers likely to BUY) ===

1. age_SENIOR, children_1-2, income_HIGH  ->  pep_YES
   support = 0.1050   confidence = 0.9403   lift = 2.0590  <-- confidence >=
0.70 AND lift > 1

2. children_1-2, income_HIGH, mortgage_NO  ->  pep_YES
   support = 0.1033   confidence = 0.9394   lift = 2.0571  <-- confidence >=
0.70 AND lift > 1

3. children_1-2, income_HIGH, save_act_YES  ->  pep_YES
   support = 0.1250   confidence = 0.9259   lift = 2.0276  <-- confidence >=
0.70 AND lift > 1

4. children_1-2, income_HIGH  ->  pep_YES
   support = 0.1450   confidence = 0.9255   lift = 2.0267  <-- confidence >=
0.70 AND lift > 1

5. age_SENIOR, children_1-2  ->  pep_YES
   support = 0.1250   confidence = 0.7979   lift = 1.7472  <-- confidence >=
0.70 AND lift > 1

6. age_SENIOR, children_1-2, save_act_YES  ->  pep_YES
```

```
   support = 0.1017   confidence = 0.7821   lift = 1.7125  <-- confidence >=
0.70 AND lift > 1

7. married_NO, mortgage_NO, save_act_YES  ->  pep_YES
   support = 0.1067   confidence = 0.7442   lift = 1.6296  <-- confidence >=
0.70 AND lift > 1

8. married_NO, mortgage_NO  ->  pep_YES
   support = 0.1533   confidence = 0.7077   lift = 1.5497  <-- confidence >=
0.70 AND lift > 1


=== RULES THAT PREDICT pep_NO (customers unlikely to buy) ===

1. children_0, married_YES, mortgage_NO, save_act_YES  ->  pep_NO
   support = 0.1217   confidence = 0.9125   lift = 1.6794

2. children_0, married_YES, save_act_YES  ->  pep_NO
   support = 0.1783   confidence = 0.8992   lift = 1.6549

3. children_0, married_YES, mortgage_NO  ->  pep_NO
   support = 0.1733   confidence = 0.8966   lift = 1.6501

4. car_NO, children_0, married_YES, mortgage_NO  ->  pep_NO
   support = 0.1000   confidence = 0.8955   lift = 1.6482

5. car_NO, children_0, married_YES  ->  pep_NO
   support = 0.1333   confidence = 0.8000   lift = 1.4724
```

2.3.1 B3 Answer — Association rules

Rule	Support	Confidence	Lift
age_SENIOR, children_1-2, income_HIGH → pep_YES	0.1050	0.9403	2.0590
children_1-2, income_HIGH, mortgage_NO → pep_YES	0.1033	0.9394	2.0571
children_1-2, income_HIGH, save_act_YES → pep_YES	0.1250	0.9259	2.0276

Rule	Support	Confidence	Lift
children_1-2, income_HIGH → pep_YES	0.1450	0.9255	2.0267
age_SENIOR, children_1-2 → pep_YES	0.1250	0.7979	1.7472
married_NO, mortgage_NO → pep_YES	0.1533	0.7077	1.5497
children_0, married_YES, save_act_YES → pep_NO	0.1783	0.8992	1.6549
children_0, married_YES → pep_NO	0.2350	0.7833	1.4417

Base rates for comparison: pep_YES = 0.4567, pep_NO = 0.5433. A lift of 2.06 means the customer group is 2.06× more likely to buy PEP than an average customer.

2.4 B4. Interpretation of the 2 Most Interesting Rules

2.4.1 Rule 1 — `children_1-2 & income_HIGH → pep_YES`

support = 0.1450, confidence = 0.9255, lift = 2.0267

Interpretation: Of the customers who have 1–2 children and are in the top third of income 92.6% bought the PEP product. That group is 14.5% of customers, and they are 2.03 times more likely to buy than an average customer.

this may occur because a household with a small number of dependants and clearly above-average income. They have real surplus income after living costs, and having children gives them a concrete long-term reason to invest — education and family security.

2.4.2 Rule 2 — `children_0 & married_YES → pep_NO`

support = 0.2350, confidence = 0.7833, lift = 1.4417

Interpretation: Of the married customers with no children, 78.3% did NOT buy PEP. This group is 23.5% of customers and is 1.44 times more likely to refuse than an average customer.

this may occur because The married couple with no dependants a dual-income household with no education or family-security pressure, and often an existing savings account that already covers their needs.